

Logging k8s cho các ứng dụng PHP

huynt93

1. Request logging

- Các ứng dụng PHP đã có access log tự động từ nginx. Vấn đề chỉ còn là bổ sung thêm một số trường dữ liệu theo yêu cầu

1. Request logging

- Giải pháp: Các ứng dụng PHP (Laravel) sẽ viết một middleware. Middleware này sẽ parse thông tin từ JWT token của GHTKID, sau đó trả ra trong response header của HTTP request.
- Response trước khi tới client sẽ được Infra xử lý, lấy thêm các thông tin đã trả ra, sau đó thêm vào k8s log trước khi kéo lên ELK. Các header này sẽ bị loại đi trước khi đến client (Client sẽ không thấy các header mới này).



php UserLoggingMiddleware.php

```
<?php

class UserLoggingMiddleware
{
    public function handle(Request $request, Closure $next): Response
    {
        $bearerToken = $request->bearerToken();
        $tokenData = explode(".", $bearerToken ?? '');
        $jwtPayload = base64_decode($tokenData[1] ?? '');
        $jwtData = json_decode($jwtPayload, true);
        $clientId = $jwtData['client_id'] ?? '';

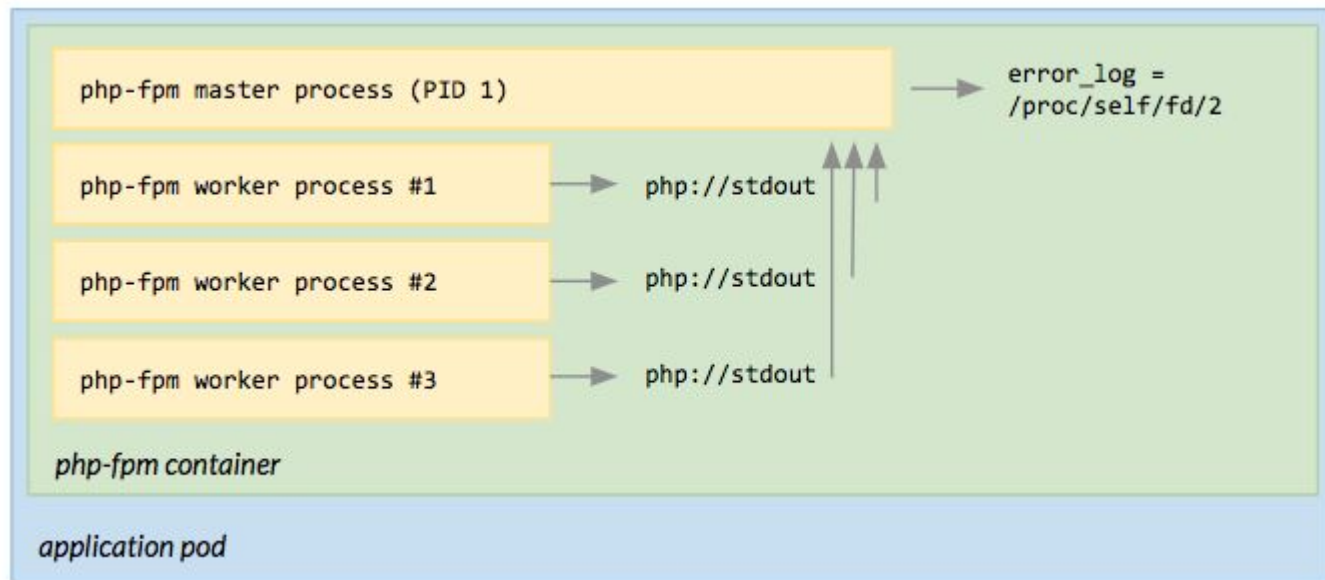
        $userId = (Auth::user())?->getGhtkId() ?? $clientId;
        $response = $next($request);
        $response->headers->set('uid', $userId);
        $response->headers->set('username', Auth::user()?->getUsername() ?? 'be-be');

        return $response;
    }
}
```

2. Application logging

Có 2 cách tiếp cận với Application Logging trong PHP:

Cách 1: Sử dụng trực tiếp `php://stdout` và `php://stderr` của PHP. Trên Laravel có thể config dễ dàng thông qua `config/logging.php` (Laravel đã sử dụng Monolog làm thư viện log tiêu chuẩn).



2. Application logging

Cách 1: Sử dụng trực tiếp `php://stdout` và `php://stderr` của PHP (log ra `STDOUT` và `STDERR` của Linux). Trên Laravel có thể config dễ dàng thông qua `config/logging.php` (Laravel đã sử dụng Monolog làm thư viện log tiêu chuẩn).

- Điểm mạnh: Tích hợp nhanh, dễ dàng.
- Điểm yếu: Bản chất các luồng `php://stdout` và `php://stderr` này được đẩy ra từ PHP-FPM, dẫn tới việc bị giới hạn độ dài, wrap bởi PHP-FPM, cũng như bị trộn lẫn log khác từ PHP-FPM.

Log bị wrap bởi PHP-FPM (có thể tắt được việc này)



logs.txt

```
[24-Jan-2019 08:30:14] WARNING: [pool www] child 6 said into stdout: "App log message to stdout"  
[24-Jan-2019 08:30:14] WARNING: [pool www] child 6 said into stderr: "App log message to stderr"
```



Codelmage

Log bị truncate tối đa 1024 ký tự bởi FPM (có thể config mở rộng được)

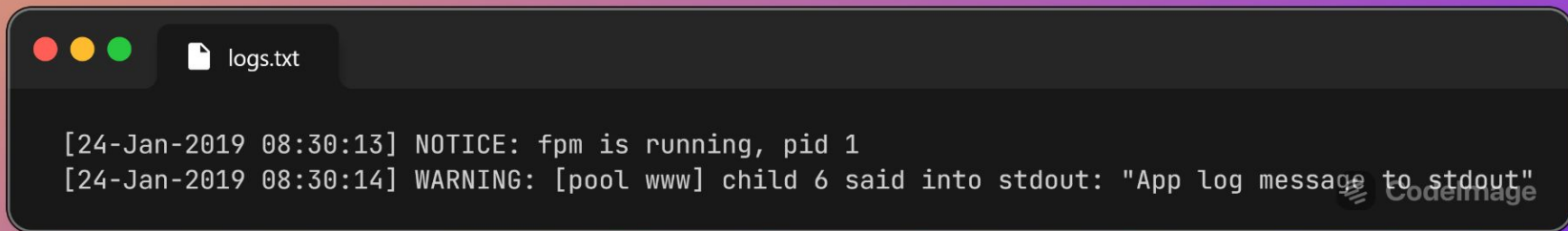


logs.txt

```
[24-Jan-2019 08:35:34] WARNING: [pool www] child 7 said into stdout: "[...] TRUNCATED"  
[24-Jan-2019 08:35:34] WARNING: [pool www] child 7 said into stdout: "CONTINUE-PREVIOUS-MESSAGE"
```

 CodeImage

Log bị trộn lẫn với log của FPM (không quá nhiều)

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and a file icon labeled 'logs.txt'. The terminal displays two lines of log output in a monospaced font. The first line is a notice from FPM, and the second line is a warning from the application pool. A faint 'CodeImage' watermark is visible in the bottom right corner of the terminal area.

```
[24-Jan-2019 08:30:13] NOTICE: fpm is running, pid 1  
[24-Jan-2019 08:30:14] WARNING: [pool www] child 6 said into stdout: "App log message to stdout"
```

2. Application logging

Cách 2: Sử dụng thêm một sidecar container chạy lệnh tail -f log

Bước 1: Mount một phân vùng dữ liệu dùng chung cho ứng dụng

Bước 2: Chạy sidecar container tail -f và kéo log sang stdout

- Điểm mạnh: Độc lập với PHP-FPM.
- Điểm yếu: Setup phức tạp với cả infra và dự án.

